Plan de Arhitectură și Implementare: AI-Augmented Collaborative Workspace

Partea I: Caiet de Sarcini, Design și Specificații Funcționale

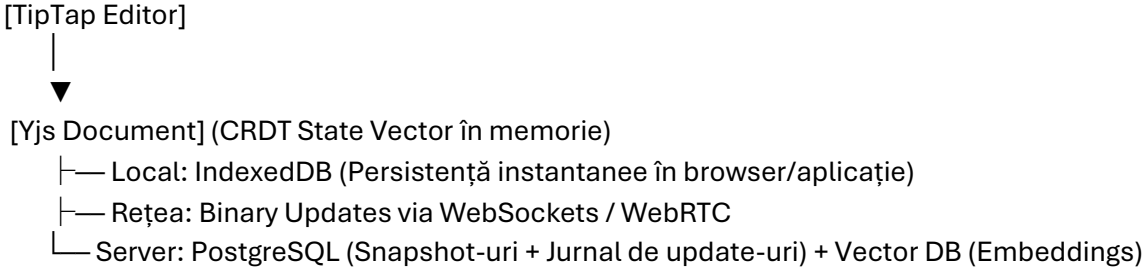
1. Ghid de Design și Ecrane (Figma & UI)

Designul se bazează pe sistemul minimalist „distraction-free” (inspirație Notion + Linear), construit modular cu kitul **Untitled UI**:

- **Ecranul 1: Autentificare / Onboarding:** Formular simplu de login/register, selecție spațiu de lucru inițial (Personal / Facultate / Proiect).
- **Ecranul 2: Workspace Principal (Layout în 3 Coloane):**
  - *Sidebar stânga (Collapsible):* Arbore de directoare (Workspaces $\rightarrow$ Caiete $\rightarrow$ Pagini), bară de căutare rapidă, comutator modul Offline/P2P, buton Graph View.
  - *Zona centrală (Editor Canvas):* Pagină curată, titlu dinamic, icon/cover opțional, spațiu de editare block-based. Meniul de formatare apare doar la selecția textului (Bubble Menu) sau la comanda / (Slash Command Menu).
  - *Sidebar dreapta (AI Co-pilot - Retractabil):* Tab-uri pentru: Chat RAG cu notițele, To-Do list extras automat de AI, sumarizări rapide.
- **Ecranul 3: Knowledge Graph View:** O planșă interactivă (canvas 2D) cu noduri (pagini/concepte) și muchii (legături semantice determinate de AI), cu filtre pe tag-uri și dată.
- **Ecranul 4: Modal Sincronizare P2P / Proximitate:** Radar vizual simplu care scanează dispozitivele din jur (via WebRTC/local Wi-Fi) și arată statusul conexiunii directe (ex. „Conectat la Tableta X – Sincronizat local”).

2. Cum se Stochează Notițele (Modelul de Date)

Pentru a permite sincronizarea instantanee fără conflicte și funcționarea offline, stocarea adoptă o strategie hibridă:



1. **Formatul de lucru (CRDT / Yjs Binary):** Notița nu se salvează clasic ca un simplu șir lung de text sau JSON la fiecare literă. Yjs reține un arbore de operații binare

(inserări/ștergeri cu ID-uri unice per utilizator). Acest binar se scrie instant în **IndexedDB** pe dispozitivul local (zero latență).

2. **Formatul de persistență pe Server (PostgreSQL):**

- Tabela documents: id, user_id, title, content_json (pentru backup și render rapid), ydoc_state (format binar bytea pentru reconstrucția stării CRDT), created_at, updated_at.
- Tabela tasks: id, document_id, text, status, deadline.

3. **Formatul Vectorial (Pinecone / Qdrant):**

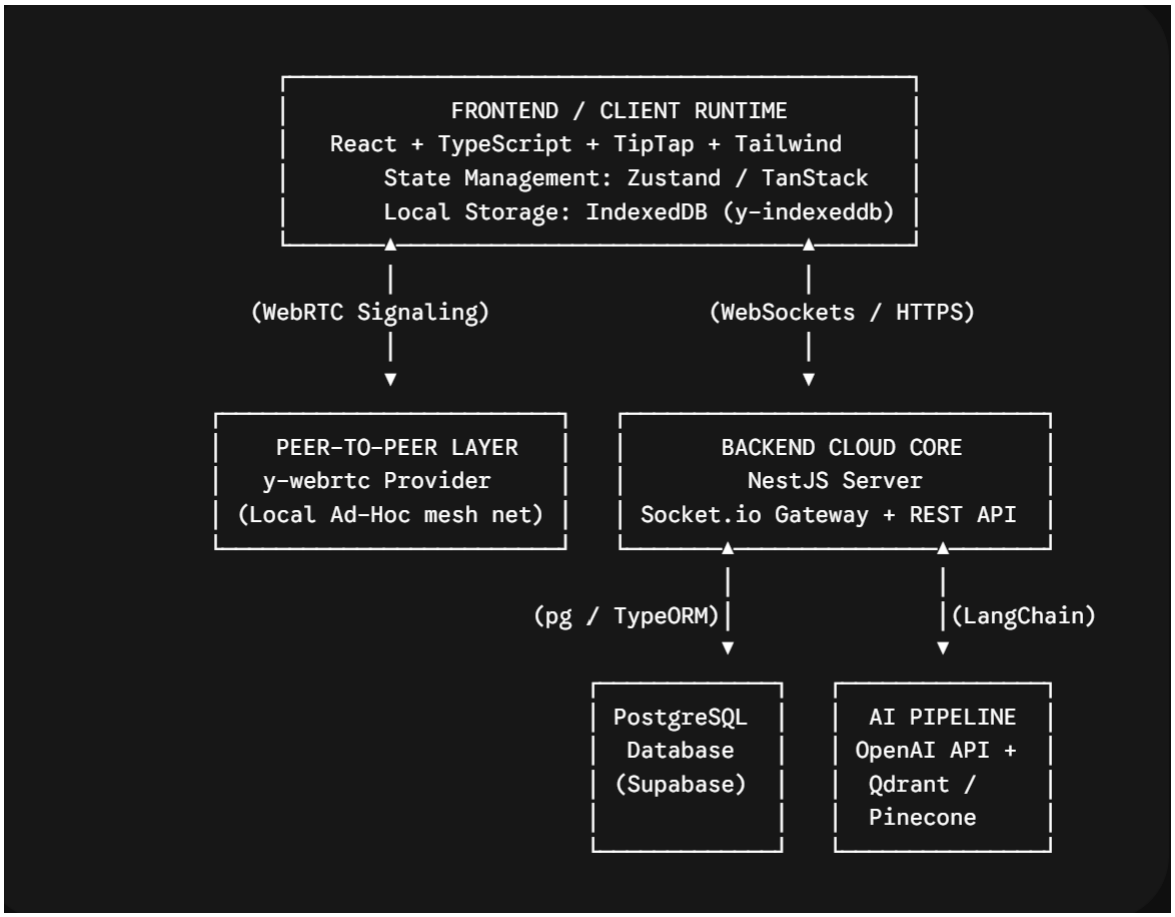
- Fragmentele de text (chunks de 200–500 tokeni) din fiecare pagină sunt convertite în vectori (1536 dimensiuni) asociați cu document_id și chunk_id.

3. Matricea Completă a Funcționalităților

Categorie	Funcționalitate	Descriere Tehnică & Comportament
Core Editor	Block-Based Editing	Suport pentru H1-H3, to-do lists, quote blocks, syntax highlighter pentru cod, tabele și imagini.
Core Editor	Slash Command & Floating Toolbar	Meniu acționat prin / pentru inserare rapidă de elemente; bară flotantă pentru bold, italic, code inline la selecție.
Ierarhie	Structură Nelimitată	Nested pages (pagină în pagină), drag-and-drop în sidebar pentru reordonare.
Sync	Real-Time Multi-Cursor	Afișarea cursorului celuilalt utilizator cu nume și culoare distinctă când se editează simultan.
Sync	Offline-First	Aplicația este complet funcțională fără internet; modificările se pun în coada IndexedDB și se propagă la reconectare.
P2P Mode	Offline Direct Sync	La lipsa semnalului extern, device-urile conectate la același hotspot/rețea locală fac schimb direct de update-uri CRDT via WebRTC.
AI Layer	Chat RAG cu Notițele	Panou unde utilizatorul întreabă ceva, sistemul extrage fragmentele relevante prin vector search și răspunde strict pe baza bazei proprii de date.

AI Layer	Auto-Tagging & Knowledge Graph	NLP-ul extrage cuvinte cheie și entități, construind automat legături între pagini care tratează subiecte similare.
AI Layer	Smart Action Items	La tastarea unei fraze de tip acțiune („Trebuie terminat referatul până joi”), AI-ul extrage automat un task în panoul To-Do cu scadența calculată.
Export	Format Universal	Export curat în fișiere .md (Markdown cu frontmatter) sau generare de PDF direct din browser/desktop.

Partea a II-a: Arhitectura Tehnologică și Ghidul de Implementare



1. Frontend & Desktop Shell

- **React (v18+) + TypeScript:** Framework-ul de bază configurat cu reguli stricte de tipizare pentru a garanta robustețea codului cerută la comisie.
- **TipTap Editor (Headless Wrapper peste ProseMirror):** Alegerea optimă pentru că separă complet logica de editare de componentele de UI. Permite maparea directă a butoanelor din *Untitled UI* pe funcțiile editorului.

- **Yjs Ecosystem (Frontend):**
  - yjs: Structura de date de bază.
  - y-prosemirror: Conectorul dintre structura de date Yjs și editorul TipTap.
  - y-indexeddb: Salvează automat orice modificare a documentului local în IndexedDB, oferind disponibilitate instantanee la reîncărcarea paginii.
- **Desktop Shell (Tauri):** Împachetează interfața web într-o aplicație de desktop nativă, utilizând motorul de randare al sistemului de operare (Webview2 pe Windows). Consumă zeci de megabytes de RAM spre deosebire de Electron (care ar consuma peste 500 MB).

2. Backend & Comunicație

- **NestJS (Node.js framework):** Arhitectură modulară enterprise (Controllers, Services, Modules, Dependency Injection), apreciată academic pentru organizare curată.
- **WebSockets (Socket.io / ws):** Modulul dedicat pentru traficul CRDT. Serverul acționează ca un releu („relay server”): primește update-ul binar de la un client și îl difuzează instantaneu tuturor clienților conectați în aceeași cameră (document ID).
- **P2P Signaling & Provider (y-webrtc):** Permite clienților să schimbe mesaje de negociere a conexiunii (SDP / ICE candidates), după care traficul de scriere curge exclusiv între dispozitive, ocolind complet serverul.

3. Pipeline-ul de Inteligență Artificială (RAG)

1. **Ingestion Worker:** Când un document este inactiv timp de 30 de secunde (debounced), backend-ul citește textul actualizat.
2. **Chunking & Embeddings:** Textul este spart în fragmente și trimis către modelul text-embedding-3-small (OpenAI), obținând reprezentarea vectorială a contextului.
3. **Vector Store (Qdrant / Pinecone):** Vectorii sunt indexați cu payload ce conține documentId și textul brut.
4. **Generare Răspuns (LangChain / LlamaIndex):** La întrebarea utilizatorului din tab-ul de AI:
   - Întrebarea este convertită în vector.
   - Baza vectorială calculează *Cosine Similarity* și returnează primele 3 cele mai relevante fragmente din notițe.
   - Fragmentele sunt injectate într-un prompt de sistem către GPT-4o-mini: *„Răspunde la întrebarea utilizatorului folosind strict contextul furnizat mai jos din notițele sale.”*